QUENYX VOPS HUB

CONFIDENTIAL — ENGINEERING

AI Adapter Developer Guide

Quenyx vOPS HUB — v1.0.0 RC1 · Document v1.0

Document Version	1.0
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Confidential — Engineering
Owner	AI Platform Engineering
Status	Released
Last Updated	2026-06-30
Document Type	Developer guide

REVISION HISTORY

Version	Date	Notes
1.0	2026-06-30	Initial guide for the AI Adapter Platform (Sprint 22): adapter lifecycle, registration, capabilities, actions, context, best practices, and a worked example for adding a new module.

Table of Contents

23 — AI Adapter Developer Guide

Audience: Module engineers adding AI to a Quenyx module. **Scope:** How to make any module (QynRun, QynNotify, QynReact, QynKnow, QynSupport, QynBalance, QynVA, …) an AI consumer using the shared **AI Adapter Platform** — without duplicating any AI logic.

---

1. Mental model

Quenyx AI never branches on module names. Every module exposes its AI surface through an **adapter**; the **registry** discovers it; the **shared narrator** does the one and only provider call.

```
Quenyx AI
  → AI Adapter Registry          (discovery: modules, capabilities, actions, entities)
    → Your Module Adapter        (metadata + capabilities + actions + buildContext)
      → Your domain services     (deterministic, real-data evidence)
        → ModuleAiNarrator       (the SINGLE provider-calling point)
          → Shared AI Platform   (provider registry, prompt orchestrator, audit, conversations)
```

Two production adapters ship today: **QynSight** (Operations Intelligence, Sprint 21) and **QynAsset** (Asset Intelligence, Sprint 22). Use them as references (`app/Services/QuenyxAI/Adapters/`).

2. The contract

`App\Contracts\QuenyxAI\AiModuleAdapter` is the coarse, discovery-oriented contract:

Method	Purpose
`moduleKey(): string`	Stable id, equal to the entitlement key (e.g. `qynrun`).
`moduleName(): string`	Human-readable name.
`moduleDescription(): string`	One-line description of the AI surface.
`moduleCategory(): string`	Functional category (e.g. `Operations`, `Asset Management`).
`moduleVersion(): string`	Adapter semantic version.
`moduleIcon(): string`	Renderer-agnostic icon key.
`capabilities(): string[]`	Machine-readable capability keys.
`supportedEntities(): string[]`	Entity types your actions operate on.
`supportedSkills(): string[]`	Shared skills you rely on (`[]` = use all).
`supportedProviders(): string[]`	Providers you pin (`[]` = platform default — the norm).
`availableActions(): array[]`	Contextual actions → UUID-only endpoints.
`buildContext(Project, options): array`	Deterministic, real-data context (no provider call).

> **Note:** this is the companion to the fine-grained `AiModuleAdapterInterface` (the 3-stage context → reasoning → prompt pipeline). The coarse contract answers "*what intelligence does this module expose*"; the fine-grained one answers "*how is one request turned into a prompt*". Most new modules only need the coarse contract plus `ModuleAiNarrator`.

2.1 Backward compatibility

Extend `App\Services\QuenyxAI\Adapters\AbstractAiModuleAdapter`. It supplies safe defaults for the metadata + `supportedEntities/Skills/Providers` methods, so you implement only what you need and the contract can grow without breaking existing adapters. (This is exactly how Sprint 21's `QynSightAiAdapter` gained metadata in Sprint 22 with **no behavior change**.)

3. Lifecycle

1. **Implement** an adapter extending `AbstractAiModuleAdapter`.
2. **Register** it once in `App\Providers\AppServiceProvider::boot()`:

```
$registry = $this->app->make(AiModuleAdapterRegistry::class);
$registry->register($this->app->make(QynRunAiAdapter::class));
```

3. **Done.** Discovery APIs, capability/action aggregation, and the AI Workspace pick it up automatically. No platform code changes.

4. Capabilities

`capabilities()` returns stable, snake_case keys (e.g. `asset_discovery_intelligence`). They are machine-readable identifiers used to group actions and drive navigation. Keep them stable once published; add new ones rather than renaming.

5. Actions

`availableActions()` returns contextual "✨ Quenyx AI" actions. Each action declares:

```
[
  'key' ⇒ 'explain_asset',                 // unique within the module
  'capability' ⇒ 'asset_discovery_intelligence',
  'target' ⇒ 'asset',                      // entity type (see supportedEntities)
  'label' ⇒ 'Explain',                     // short verb for the UI
  'method' ⇒ 'POST',
  'endpoint' ⇒ '/api/qynasset/intelligence/assets/{uuid}/explain', // UUID-only, workspace-scop
],
```

Rules: endpoints are **workspace-scoped** (`?workspace=` / body) and **UUID-only** — never expose a numeric id. If your entity has numeric keys, derive a deterministic UUIDv5 (see §7).

6. Context

`buildContext(Project $project, array $options = [])` returns a deterministic snapshot built from **real domain data only**, plus grounding guardrails. It **must not** call a provider. When evidence is missing, say so in the context (e.g. `available ⇒ false`) — never fabricate. The shared narrator turns this context into prose.

7. UUID-only entity ids

Reuse the deterministic UUIDv5 pattern (`Ramsey\Uuid::uuid5`):

```
Uuid::uuid5(Uuid::NAMESPACE_URL, "quenyx://{$module}/{$type}/{$workspaceId}/{$id}")→toString();
```

Provide a resolver that scans the small, workspace-scoped candidate set to map a UUID back to the row. See `App\Support\Asset\AssetEntityId` + `AssetEntityResolver` (and the QynSight equivalents).

8. Narrating with the shared runtime

Never call a provider directly. Inject `App\Services\Ai\ModuleAiNarrator` and call `narrate()`:

```
$ai = $this->narrator->narrate(
    $project, $user,
    'my_context_type',
    $evidence,                        // real, deterministic data
    $question,                        // grounded instruction
    self::ROLE_PREAMBLE,              // "Use ONLY the evidence; never fabricate…"
    'mymodule_intelligence_explain', // fully-qualified audit action
    'mymodule.intelligence.explain', // endpoint label for audit
    ModuleAiNarrator::DEFAULT_GUARDRAILS,
    'text',
    $citations,                       // evidence references the model must cite
);
```

The narrator handles provider resolution (mock when AI is disabled, so the surface is always production-safe), grounded prompt assembly, audit, and a stable response envelope. For chat-style features, also reuse `AiConversationRepository` so threads appear in Quenyx AI conversations (see `AssetIntelligenceService::copilot`).

9. Security checklist

Every module AI endpoint must be: workspace-aware (required `workspace` UUID), entitlement-gated (your `moduleKey`), RBAC-gated (`ProjectPolicy::accessAi`), capability-gated (`can_use_ai` for AI actions), audited, provider-logged, conversation-logged (for chat), rate-limited (`throttle:ai-workspace`), and UUID-only. Use a base controller (see `AssetIntelligenceBaseController`) to centralise this envelope.

10. Best practices

- **Reuse, don't duplicate.** No new provider registry, prompt engine, reasoning engine, RAG engine, or chat UI. Reuse platform services and your own module's domain services.
- **Evidence first.** Business logic decides *what* (deterministic); AI only *renders*. Every recommendation cites evidence.
- **Be honest about gaps.** If a capability has no data source, expose it but report `available ⇒ false` with the integration required — never fabricate.
- **Stable ids & keys.** UUID-only entities; stable capability/action keys.
- **i18n.** Provide EN + AR for any new UI strings.

11. Worked example — adding "QynRun" AI (sketch)

1. `app/Services/QuenyxAI/Adapters/QynRunAiAdapter.php` extends `AbstractAiModuleAdapter`; declares `moduleKey() = 'qynrun'`, capabilities (e.g. `job_intelligence`, `pipeline_health`), actions, and `buildContext()` from QynRun's real job/run data.
2. `app/Services/Run/Intelligence/RunEvidenceCollector.php` + feature services build deterministic evidence; `RunIntelligenceService` narrates via `ModuleAiNarrator`.

3. `app/Http/Controllers/Run/Intelligence/*` + `routes/qynrun-intelligence.php` expose UUID-only, workspace-scoped endpoints behind the standard envelope; include the route file from `api.php`.
4. Register `QynRunAiAdapter` in `AppServiceProvider::boot()`.
5. Frontend: a `runIntelligenceService.ts`, a dashboard page, and contextual buttons reusing the generic `AiCopilotDrawer`. Add EN/AR strings.

That's it — the discovery APIs, capability aggregation, and AI Workspace surface QynRun automatically, with no platform changes.

12. Reference files

- Contract: `app/Contracts/QuenyxAI/AiModuleAdapter.php`
- Base: `app/Services/QuenyxAI/Adapters/AbstractAiModuleAdapter.php`
- Registry: `app/Services/QuenyxAI/AiModuleAdapterRegistry.php`
- Shared narrator: `app/Services/Ai/ModuleAiNarrator.php`
- Discovery API: `app/Http/Controllers/Ai/Workspace/AiAdapterController.php`
- Example adapters: `QynSightAiAdapter`, `QynAssetAiAdapter`
- Example module: `app/Services/Asset/Intelligence/*`, `routes/qynasset-intelligence.php`